

Experiment: Library Management System

Java Program Demonstrating Object-Oriented Programming Concepts

1. Question

Develop a Java-based "Library Management System" to demonstrate Object-Oriented Programming concepts. Implement the following:

Part A: Classes and Objects

- Create a Book class containing Book ID, Book Name, Author, and Price.
- Create objects and display their details.

Part B: Constructors and Encapsulation

- Create default and parameterized constructors.
- Declare data members as private.
- Access the data members using Getter and Setter methods.

Part C: Inheritance

- Create a Person class.
- Create Student and Faculty classes that inherit from Person.
- Display the inherited properties.

Part D: Polymorphism

- Implement method overloading for calculating the area of different shapes.
- Implement method overriding using Vehicle, Car, and Bike classes.

Part E: Abstraction and Interfaces

- Create an abstract Shape class with an abstract draw() method.
- Implement draw() in Circle and Rectangle.
- Create a Printable interface and implement it in the Report class.

2. Steps

- 1 Start the program.
- 2 Create the Book class with private data members bookId, bookName, author, and price.
- 3 Create a default constructor for Book.
- 4 Create a parameterized constructor for Book.
- 5 Create Getter and Setter methods for all Book data members.
- 6 Create the displayBookDetails() method.
- 7 Create Book objects using both constructors.
- 8 Use Setter methods to assign values to the first book.
- 9 Use Getter methods to access the second book's details.
- 10 Create the Person parent class with name and age.
- 11 Create the Student class extending Person.
- 12 Create the Faculty class extending Person.
- 13 Create Student and Faculty objects and display their inherited properties.
- 14 Create the AreaCalculator class.
- 15 Create overloaded area() methods for circle, rectangle, square, and triangle.
- 16 Call the overloaded methods with different arguments.
- 17 Create the Vehicle parent class with the display() method.
- 18 Create Car and Bike classes and override the display() method.

- 19 Use parent-class references to demonstrate runtime polymorphism.
- 20 Create the abstract Shape class with the abstract draw() method.
- 21 Create Circle and Rectangle classes extending Shape.
- 22 Implement the draw() method in both classes.
- 23 Create the Printable interface with the print() method.
- 24 Create the Report class implementing Printable.
- 25 Create a Report object using an interface reference.
- 26 Call the print() method.
- 27 Display the successful execution message.
- 28 Stop the program.

3. Algorithm

Part A & B – Classes, Objects, Constructors and Encapsulation

- 1 Start.
- 2 Define the Book class.
- 3 Declare bookId, bookName, author, and price as private.
- 4 Define a default constructor.
- 5 Define a parameterized constructor.
- 6 Define Getter and Setter methods.
- 7 Create book1 using the default constructor.
- 8 Set its values using Setter methods.
- 9 Display its details.
- 10 Create book2 using the parameterized constructor.
- 11 Display its details.
- 12 Access book2 using Getter methods.

Part C – Inheritance

- 1 Define the Person parent class.
- 2 Declare name and age.
- 3 Create the Student class extending Person.
- 4 Create the Faculty class extending Person.
- 5 Initialize parent properties using super().
- 6 Create a Student object and display its details.
- 7 Create a Faculty object and display its details.

Part D – Method Overloading

- 1 Create the AreaCalculator class.
- 2 Define area(double radius) for a circle.
- 3 Define area(double length, double breadth) for a rectangle.
- 4 Define area(int side) for a square.
- 5 Define area(double base, double height, boolean triangle) for a triangle.
- 6 Call each overloaded method.
- 7 Display the calculated areas.

Part D – Method Overriding

- 1 Create the Vehicle parent class.
- 2 Define the display() method.
- 3 Create Car extending Vehicle.
- 4 Override display() in Car.
- 5 Create Bike extending Vehicle.
- 6 Override display() in Bike.
- 7 Create Vehicle references pointing to Car and Bike objects.
- 8 Call display().
- 9 Display the overridden results.

Part E – Abstraction

- 1 Create the abstract Shape class.
- 2 Declare abstract method draw().
- 3 Create Circle extending Shape.
- 4 Implement draw() in Circle.
- 5 Create Rectangle extending Shape.
- 6 Implement draw() in Rectangle.
- 7 Create Shape references for Circle and Rectangle.

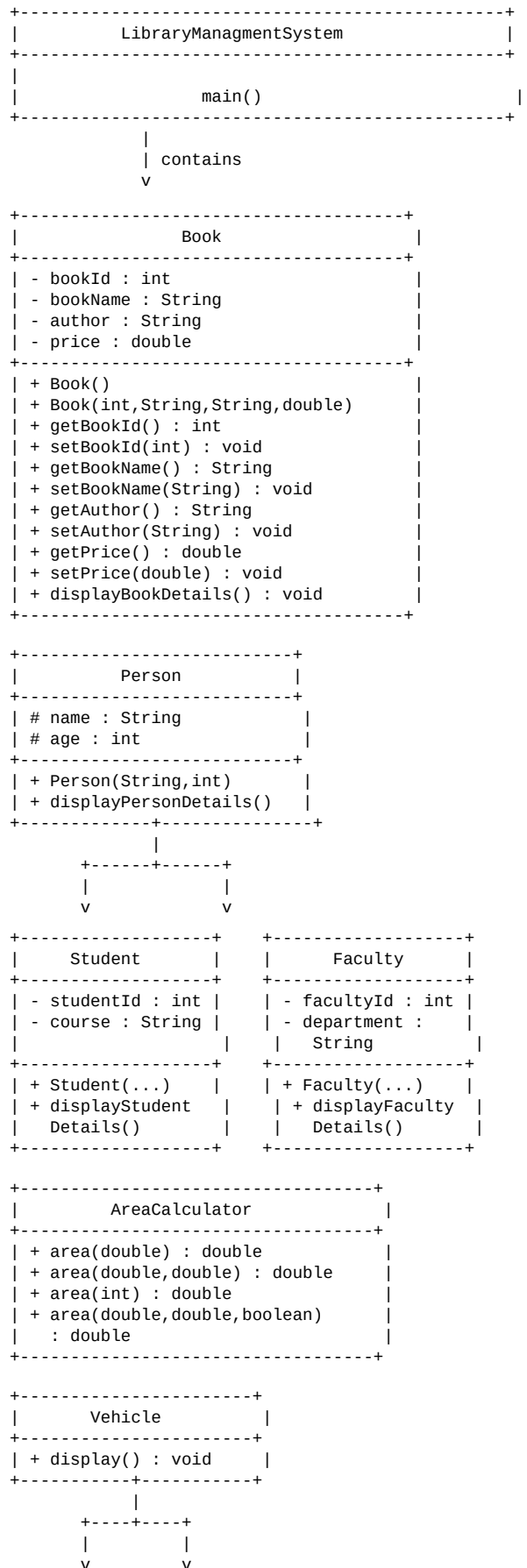
8 Call draw().

Part E – Interface

- 1 Create the Printable interface.
- 2 Declare the print() method.
- 3 Create the Report class implementing Printable.
- 4 Implement print() in Report.
- 5 Create a Printable reference pointing to a Report object.
- 6 Call print().
- 7 Stop.

4. UML Diagram

Exact UML Class Diagram for Your Code



Car	Bike
+ display()	+ display()

```

+-----+
| <<abstract>> Shape |
+-----+
| + draw() : void      |
+-----+

```

```

      |
      +-----+
      |         |
      v         v

```

Circle	Rectangle
+ draw()	+ draw()

```

+-----+
| <<interface>>      |
| Printable           |
+-----+
| + print() : void    |
+-----+

```

```

      |
      v

```

```

+-----+
| Report |
+-----+
| + print() : void |
+-----+

```

Inheritance Relationships

```
Student  -----|> Person
Faculty  -----|> Person

Car       -----|> Vehicle
Bike      -----|> Vehicle

Circle    -----|> Shape
Rectangle -----|> Shape

Report    -----|> Printable
```

Polymorphism

Method Overloading:

```
AreaCalculator
|
+-- area(double)
+-- area(double, double)
+-- area(int)
+-- area(double, double, boolean)
```

Method Overriding:

```
Vehicle
|
+-- Car  -> display()
|
+-- Bike -> display()
```

5. Code

```
package packagecodejava;
public class LibraryManagmentSystem {

    // =====
    // PART A & B: CLASS, OBJECTS, CONSTRUCTORS, ENCAPSULATION
    // =====

    static class Book {

        // Private data members - Encapsulation
        private int bookId;
        private String bookName;
        private String author;
        private double price;

        // Default constructor
        public Book() {
            bookId = 0;
            bookName = "Unknown";
            author = "Unknown";
            price = 0.0;
        }

        // Parameterized constructor
        public Book(int bookId, String bookName, String author, double price) {
            this.bookId = bookId;
            this.bookName = bookName;
            this.author = author;
            this.price = price;
        }

        // Getter and Setter for Book ID
        public int getBookId() {
            return bookId;
        }

        public void setBookId(int bookId) {
            this.bookId = bookId;
        }

        // Getter and Setter for Book Name
        public String getBookName() {
            return bookName;
        }

        public void setBookName(String bookName) {
            this.bookName = bookName;
        }

        // Getter and Setter for Author
        public String getAuthor() {
            return author;
        }

        public void setAuthor(String author) {
            this.author = author;
        }

        // Getter and Setter for Price
        public double getPrice() {
            return price;
        }

        public void setPrice(double price) {
            this.price = price;
        }

        // Method to display book details
        public void displayBookDetails() {
            System.out.println("Book ID : " + bookId);
            System.out.println("Book Name : " + bookName);
            System.out.println("Author : " + author);
            System.out.println("Price : Rs. " + price);
        }
    }
}
```



```

}

// =====
// PART C: INHERITANCE
// =====

// Parent class
static class Person {

    protected String name;
    protected int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void displayPersonDetails() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}

// Student inherits Person
static class Student extends Person {

    private int studentId;
    private String course;

    public Student(String name, int age, int studentId, String course) {
        super(name, age);
        this.studentId = studentId;
        this.course = course;
    }

    public void displayStudentDetails() {
        displayPersonDetails();
        System.out.println("Student ID : " + studentId);
        System.out.println("Course : " + course);
    }
}

// Faculty inherits Person
static class Faculty extends Person {

    private int facultyId;
    private String department;

    public Faculty(String name, int age, int facultyId, String department) {
        super(name, age);
        this.facultyId = facultyId;
        this.department = department;
    }

    public void displayFacultyDetails() {
        displayPersonDetails();
        System.out.println("Faculty ID : " + facultyId);
        System.out.println("Department : " + department);
    }
}

// =====
// PART D: METHOD OVERLOADING
// =====

static class AreaCalculator {

    // Area of circle
    public double area(double radius) {
        return Math.PI * radius * radius;
    }

    // Area of rectangle
    public double area(double length, double breadth) {

```

```

        return length * breadth;
    }

    // Area of square
    public double area(int side) {
        return side * side;
    }

    // Area of triangle
    public double area(double base, double height, boolean triangle) {
        return 0.5 * base * height;
    }
}

// =====
// PART D: METHOD OVERRIDING
// =====

// Parent Vehicle class
static class Vehicle {

    public void display() {
        System.out.println("This is a Vehicle.");
    }
}

// Car overrides display()
static class Car extends Vehicle {

    @Override
    public void display() {
        System.out.println("This is a Car.");
    }
}

// Bike overrides display()
static class Bike extends Vehicle {

    @Override
    public void display() {
        System.out.println("This is a Bike.");
    }
}

// =====
// PART E: ABSTRACTION
// =====

// Abstract class
static abstract class Shape {

    // Abstract method
    public abstract void draw();
}

// Circle implements abstract method
static class Circle extends Shape {

    @Override
    public void draw() {
        System.out.println("Drawing a Circle.");
    }
}

// Rectangle implements abstract method
static class Rectangle extends Shape {

    @Override
    public void draw() {
        System.out.println("Drawing a Rectangle.");
    }
}

// =====
// PART E: INTERFACE

```

```

// =====

interface Printable {

    void print();

}

// Report implements Printable
static class Report implements Printable {

    @Override
    public void print() {
        System.out.println("Printing the Library Report.");
    }

}

// =====
// MAIN METHOD
// =====

public static void main(String[] args) {

    System.out.println("=====");
    System.out.println("                LIBRARY MANAGEMENT SYSTEM");
    System.out.println("=====");

    // =====
    // PART A & B: CLASSES, OBJECTS, CONSTRUCTORS,
    //                ENCAPSULATION
    // =====

    System.out.println("\n\n--- PART A & B: BOOK DETAILS ---");

    // Object using default constructor
    Book book1 = new Book();

    // Using setters
    book1.setBookId(101);
    book1.setBookName("Java Programming");
    book1.setAuthor("James Gosling");
    book1.setPrice(450.00);

    System.out.println("\nBook 1 - Default Constructor + Setters");
    book1.displayBookDetails();

    // Object using parameterized constructor
    Book book2 = new Book(
        102,
        "Object Oriented Programming",
        "Robert Lafore",
        550.00
    );

    System.out.println("\nBook 2 - Parameterized Constructor");
    book2.displayBookDetails();

    // Demonstrating getters
    System.out.println("\nAccessing Book 2 using Getter Methods:");
    System.out.println("Book ID   : " + book2.getBookId());
    System.out.println("Book Name : " + book2.getBookName());
    System.out.println("Author    : " + book2.getAuthor());
    System.out.println("Price     : Rs. " + book2.getPrice());

    // =====
    // PART C: INHERITANCE
    // =====

    System.out.println("\n\n--- PART C: INHERITANCE ---");

    // Student object
    Student student = new Student(

```

```

        "Rahul",
        20,
        501,
        "Computer Science"
    );

    System.out.println("\nStudent Details:");
    student.displayStudentDetails();

    // Faculty object
    Faculty faculty = new Faculty(
        "Dr. Priya",
        40,
        301,
        "Computer Science"
    );

    System.out.println("\nFaculty Details:");
    faculty.displayFacultyDetails();

    // =====
    // PART D: METHOD OVERLOADING
    // =====

    System.out.println("\n\n--- PART D: METHOD OVERLOADING ---");

    AreaCalculator calculator = new AreaCalculator();

    // Circle
    double circleArea = calculator.area(5.0);
    System.out.println("Area of Circle = " + circleArea);

    // Rectangle
    double rectangleArea = calculator.area(10.0, 5.0);
    System.out.println("Area of Rectangle = " + rectangleArea);

    // Square
    double squareArea = calculator.area(6);
    System.out.println("Area of Square = " + squareArea);

    // Triangle
    double triangleArea = calculator.area(8.0, 4.0, true);
    System.out.println("Area of Triangle = " + triangleArea);

    // =====
    // PART D: METHOD OVERRIDING & RUNTIME POLYMORPHISM
    // =====

    System.out.println("\n\n--- METHOD OVERRIDING ---");

    // Parent reference referring to Car object
    Vehicle vehicle1 = new Car();

    // Parent reference referring to Bike object
    Vehicle vehicle2 = new Bike();

    vehicle1.display();
    vehicle2.display();

    // =====
    // PART E: ABSTRACTION
    // =====

    System.out.println("\n\n--- PART E: ABSTRACTION ---");

    // Abstract class reference
    Shape shape1 = new Circle();
    Shape shape2 = new Rectangle();

    shape1.draw();
    shape2.draw();
    // =====

```

```

// PART E: INTERFACE
// =====

System.out.println("\n\n--- INTERFACE ---");

Printable report = new Report();

report.print();

// =====
// END
// =====

System.out.println("\n=====");
System.out.println("          PROGRAM EXECUTED SUCCESSFULLY");
System.out.println("=====");
}
}

```

eclipse-workspace - packagecodejava/src/packagecodejava/LibraryManagmentSystem.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Tasks Console X

<terminated> LibraryManagmentSystem [Java Application] C:\Users\Subba Reddy\Downloads\eclipse-jee-2026-06-R-win32-x86_64\eclipse\plugins

```
=====
LIBRARY MANAGEMENT SYSTEM
=====

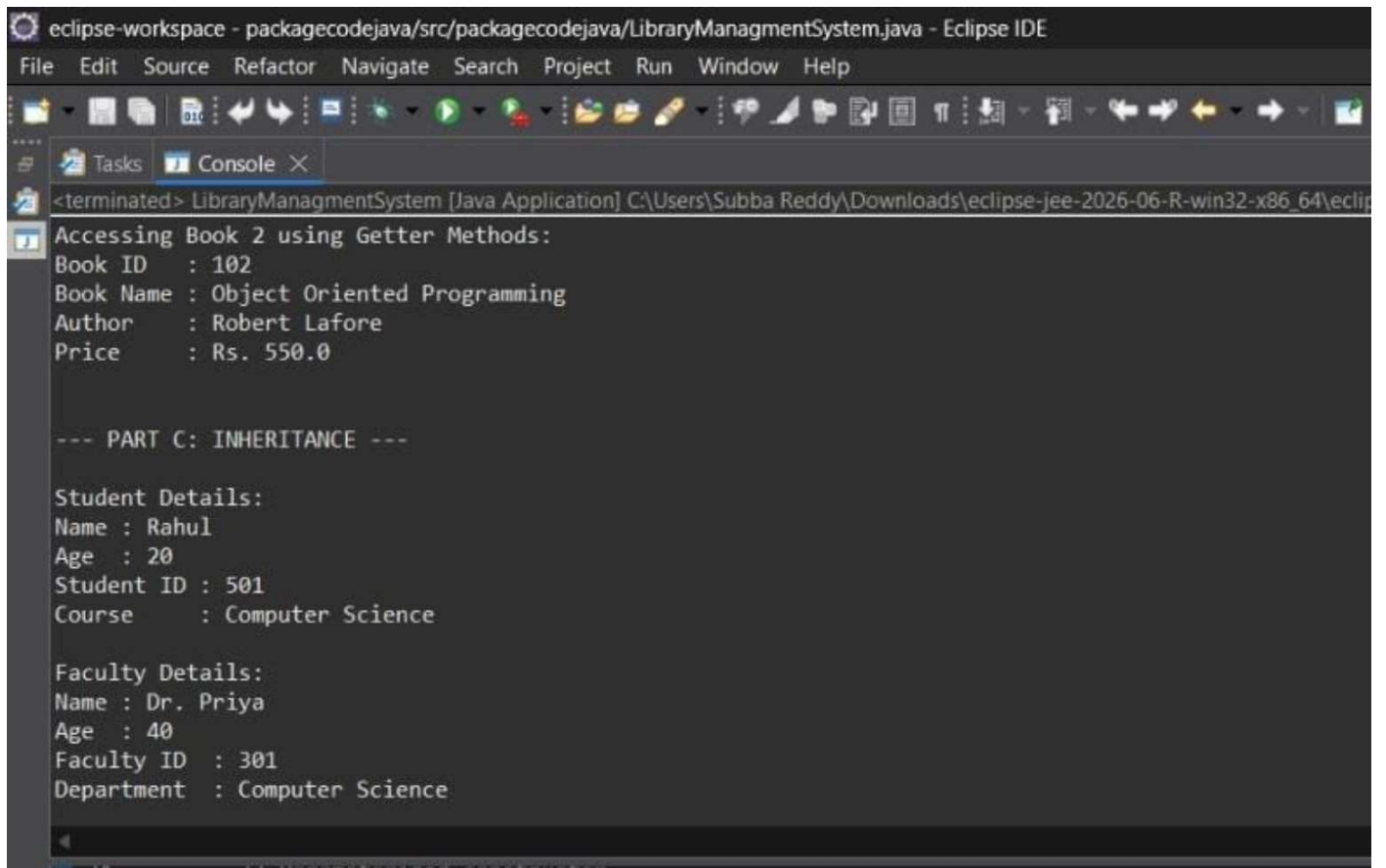
--- PART A & B: BOOK DETAILS ---

Book 1 - Default Constructor + Setters
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : Rs. 450.0

Book 2 - Parameterized Constructor
Book ID   : 102
Book Name : Object Oriented Programming
Author    : Robert Lafore
Price     : Rs. 550.0

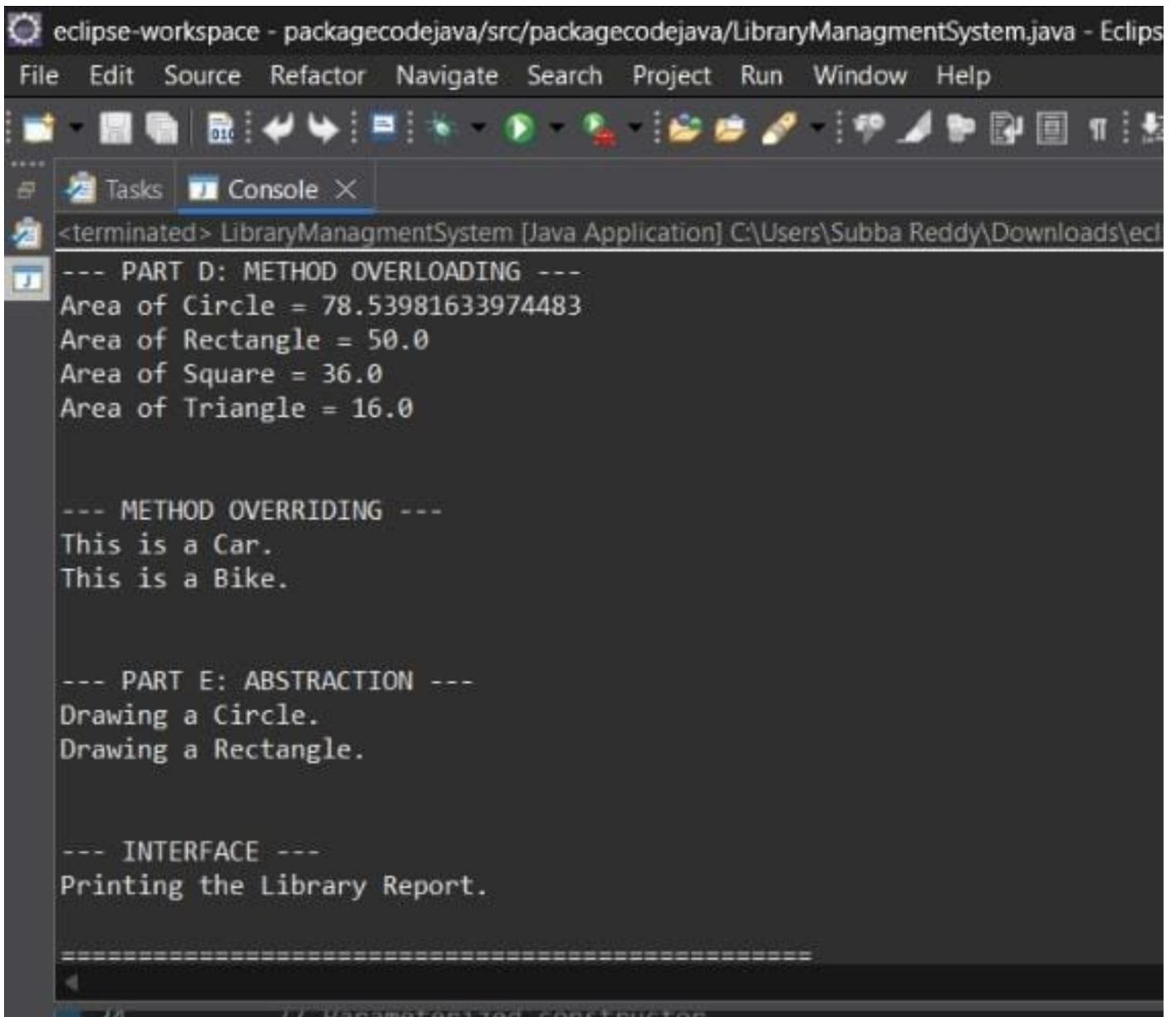
Accessing Book 2 using Getter Methods:
Book ID   : 102

24 // Parameterized constructor
25 public Book(int bookId, String bookName, String author, double price) {
26     this.bookId = bookId;
```

The image shows the Eclipse IDE interface. The title bar reads "eclipse-workspace - packagecodejava/src/packagecodejava/LibraryManagmentSystem.java - Eclipse IDE". The menu bar includes File, Edit, Source, Refactor, Navigate, Search, Project, Run, Window, and Help. The toolbar contains various icons for file operations, navigation, and execution. The "Console" tab is active, showing the following output:

```
<terminated> LibraryManagmentSystem [Java Application] C:\Users\Subba Reddy\Downloads\eclipse-jee-2026-06-R-win32-x86_64\ecli  
Accessing Book 2 using Getter Methods:  
Book ID : 102  
Book Name : Object Oriented Programming  
Author : Robert Lafore  
Price : Rs. 550.0  
  
--- PART C: INHERITANCE ---  
  
Student Details:  
Name : Rahul  
Age : 20  
Student ID : 501  
Course : Computer Science  
  
Faculty Details:  
Name : Dr. Priya  
Age : 40  
Faculty ID : 301  
Department : Computer Science
```

Program Output Screenshot



The screenshot shows the Eclipse IDE interface with the console window open. The title bar indicates the workspace is 'eclipse-workspace' and the file is 'packagecodejava/src/packagecodejava/LibraryManagmentSystem.java'. The console output is as follows:

```
<terminated> LibraryManagmentSystem [Java Application] C:\Users\Subba Reddy\Downloads\ec  
--- PART D: METHOD OVERLOADING ---  
Area of Circle = 78.53981633974483  
Area of Rectangle = 50.0  
Area of Square = 36.0  
Area of Triangle = 16.0  
  
--- METHOD OVERRIDING ---  
This is a Car.  
This is a Bike.  
  
--- PART E: ABSTRACTION ---  
Drawing a Circle.  
Drawing a Rectangle.  
  
--- INTERFACE ---  
Printing the Library Report.  
  
=====
```

At the bottom of the console, a line of code is partially visible: `// Parameterized constructor`.